Online Movie /Event Ticket Booking System

Introduction

Movie Magic is a system developed to manage ticket booking for movies and events in an organized and efficient way. It helps users easily explore available movies or events, check schedules and venues, select seats, and book tickets without confusion or delay.

The system handles important aspects such as show scheduling, seat availability, booking details, and payment records. It ensures that all information is stored properly, reducing manual errors and improving accuracy.

In this system, multiple movies or events can be available at different venues and times, and users can make bookings based on their preferences. Each booking can include one or more seats, and all booking and payment details are maintained for future reference.

Overall, Movie Magic provides a simple and reliable way to manage ticket booking activities for both users and administrators.

Objectives

To provide easy ticket booking for movies and events

To allow users to view schedules and select seats

To maintain booking and payment records

To reduce manual effort and errors

Scope

Users can view and book tickets for movies/events

The system manages schedules, seats, and venues

Booking and payment details are stored

Admin can manage movies, events, and bookings

1.1 All the entities involved in the system

- Admin
- Booked seats
- Bookings
- Events
- Movie
- Customer
- Event-bookings
- OTP-verification
- payments
- Shows
- Theater

1.2 List the attributes of each entity

1) Admin

- ADMINID
- Name
- Email
- Password
- Role

2)Booked seats

- Seat ID
- Booking ID
- Screen ID
- Seat number

3)Bookings

- BookingID
- Screen ID
- User ID
- No of seats

4) customer

- user-id
- name
- email
- password
- city

5) Event-bookings

- id
- event-id
- user-id
- user-name
- email

6) Events

- id
- event-name
- category
- city
- venue

7) Movie

- MovieID
- Movie Name
- Rating
- Dimension
- Genre

8)OTP-verifications

- OTP ID
- User ID
- Email
- expires at
- created at

9) Payments

- Payment ID
- Booking ID
- Amount
- Status
- Transaction ID
- Event booking ID

10) Shows

- Screen-id
- Movie ID
- Theater ID
- Timings
- Showdate

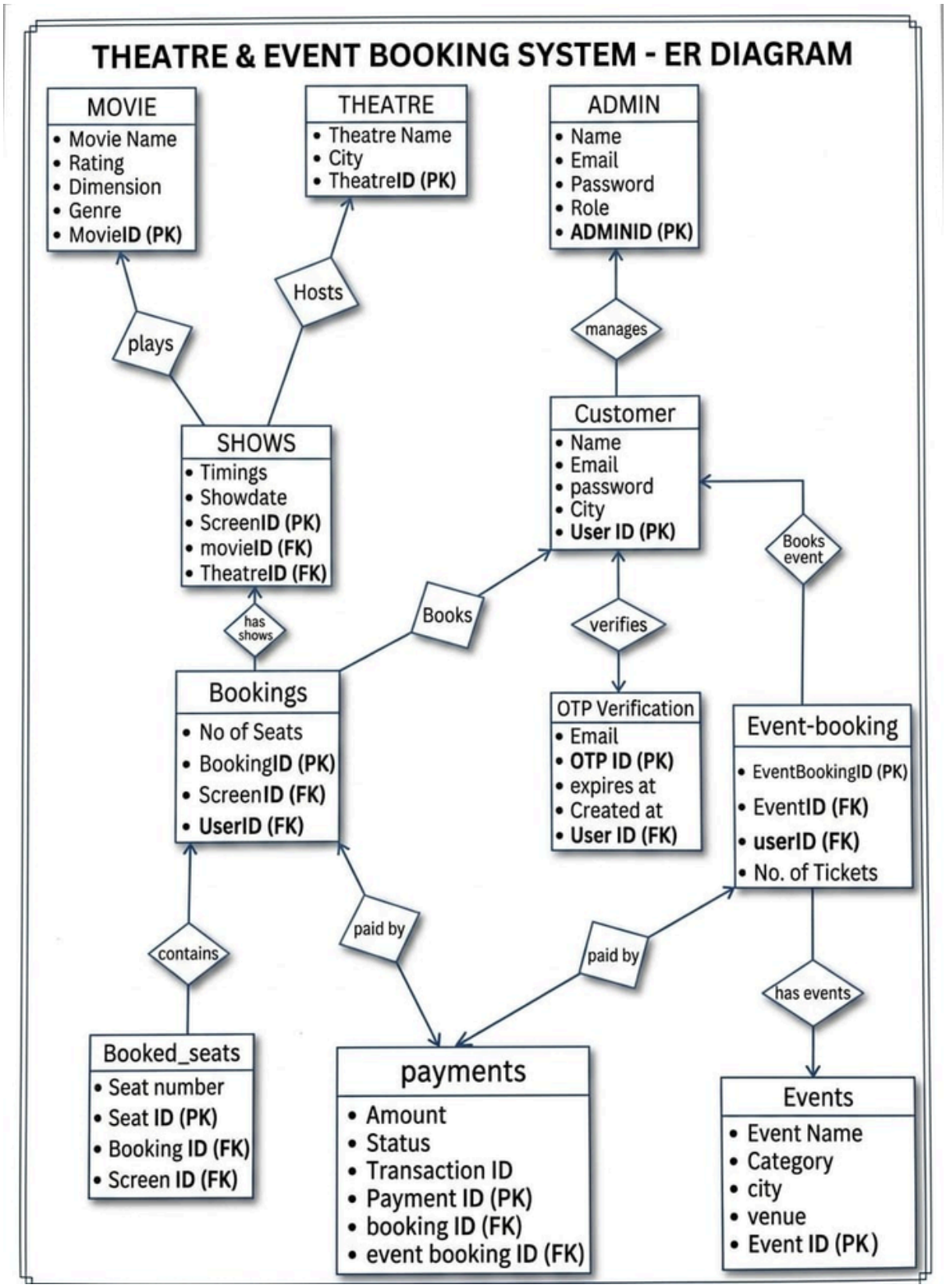
11)Theater

- TheaterID
- Theater Name
- City

1.3 Specify the cardinality of each relationship

Relationship	Cardinality
Movie-Shows	1:N
Theater-Shows	1:N
Admin-Customer	1:N
Shows-Bookings	1:N
Customer-Bookings	1:N
Customer-OTP_verification	1:1
Customer- Event_bookings	1:N
Events-Event_bookings	1:N
Bookings-Booked_seats	1:N
Bookings-Payments	1:1

1.4 E-R diagram for the system



2.1 Represent all entities and relationships in tabular format

Customer

Column	Datatype	Constraint
user_id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL
city	VARCHAR(255)	
contact_no	BIGINT	
provider	VARCHAR(50)	DEFAULT 'local'
profile_pic	TEXT	
is_blocked	BOOLEAN	DEFAULT false

Movie

Column	Datatype	Constraint
movie_id	VARCHAR(255)	PRIMARY KEY
movie_name	VARCHAR(255)	NOT NULL
movie_rating	VARCHAR(50)	
movie_dimensions	VARCHAR(50)	
genre	VARCHAR(255)	
status	VARCHAR(50)	
description	TEXT	
language	VARCHAR(100)	
category	VARCHAR(100)	
duration	VARCHAR(50)	
poster_url	VARCHAR(500)	
poster	VARCHAR(500)	

Shows

Column	Datatype	Constraint
screen_id	VARCHAR(255)	PRIMARY KEY
movie_id	VARCHAR(255)	FOREIGN KEY
theater_id	INTEGER	FOREIGN KEY
timings	TIME	NOT NULL
show_date	DATE	NOT NULL
screen_no	INTEGER	DEFAULT 1
screen_dimensions	VARCHAR(50)	
no_of_seats	INTEGER	DEFAULT 0
selected_seats	JSONB	DEFAULT []

Bookings

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
screen_id	VARCHAR(255)	FOREIGN KEY
user_id	INTEGER	
no_of_seats	INTEGER	DEFAULT 0
selected_seats	JSONB	DEFAULT []
price	NUMERIC(10,2)	DEFAULT 0.00
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
payment_status	VARCHAR(50)	DEFAULT 'Paid'
theater_id	INTEGER	
theater_name	VARCHAR(255)	
movie_id	VARCHAR(255)	
show_date	DATE	
show_time	TIME	

Booked Seats

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
booking_id	INTEGER	FOREIGN KEY
screen_id	VARCHAR(255)	FOREIGN KEY
seat_number	VARCHAR(50)	NOT NULL, UNIQUE (with screen_id)

Theater

Column	Datatype	Constraint
theater_id	SERIAL	PRIMARY KEY
theater_name	VARCHAR(255)	NOT NULL
city	VARCHAR(255)	

Payments

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
booking_id	INTEGER	FOREIGN KEY
amount	NUMERIC(10,2)	NOT NULL
status	VARCHAR(50)	DEFAULT 'SUCCESS'
transaction_id	VARCHAR(255)	
payment_method	VARCHAR(50)	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

OTP Verifications

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
email	VARCHAR(100)	NOT NULL
otp	VARCHAR(6)	NOT NULL
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
is_verified	BOOLEAN	DEFAULT false

Events

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
event_name	VARCHAR(255)	NOT NULL
category	VARCHAR(50)	NOT NULL
city	VARCHAR(100)	NOT NULL
venue	VARCHAR(255)	NOT NULL
event_date	DATE	NOT NULL
event_time	VARCHAR(20)	NOT NULL
price	INTEGER	NOT NULL

Event Bookings

Column	Datatype	Constraint
id	SERIAL	PRIMARY KEY
event_id	INTEGER	FOREIGN KEY
user_id	INTEGER	NOT NULL
user_name	VARCHAR(255)	
email	VARCHAR(255)	
phone	VARCHAR(20)	
tickets_count	INTEGER	NOT NULL
selected_seats	TEXT[]	
amount	INTEGER	NOT NULL
payment_status	VARCHAR(20)	DEFAULT 'Paid'
booking_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

3.1 Convert the unnormalized travel reservation data into normalized relations.

Unnormalised Form (UNF)

The initial data contains:

Repeating groups (Seats, Tickets)

Possible multi-valued attributes

Data redundancy

First Normal Form (1NF)

All attributes are made atomic (single-valued) and repeating groups are removed.

Separate relations are created:

BOOKINGS → BOOKED_SEATS (for multiple seats)

EVENT_BOOKING → EVENT_TICKETS (for multiple tickets)

All tables now contain only single-valued attributes.

Second Normal Form (2NF)

All relations are in 1NF and have single primary keys, so there is no partial dependency.

Therefore, all relations satisfy 2NF.

Third Normal Form (3NF)

Transitive dependency is removed.

EVENTS table is decomposed into:

EVENTS(EventID, EventName, Category, VenueID)

VENUE(VenueID, VenueName, City)

3.2 Write the SQL commands to create the normalized tables

```
CREATE TABLE CUSTOMER (
    UserID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Password VARCHAR(100),
    City VARCHAR(50)
);

CREATE TABLE MOVIE (
    MovieID INT PRIMARY KEY,
    MovieName VARCHAR(100),
    Rating FLOAT,
    Dimension VARCHAR(10),
    Genre VARCHAR(50)
);

CREATE TABLE THEATRE (
    TheatreID INT PRIMARY KEY,
    TheatreName VARCHAR(100),
```

```
    City VARCHAR(50)
);

CREATE TABLE SHOWS (
    ScreenID INT PRIMARY KEY,
    Timings VARCHAR(50),
    ShowDate DATE,
    MovieID INT,
    TheatreID INT,
    FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),
    FOREIGN KEY (TheatreID) REFERENCES THEATRE(TheatreID)
);

CREATE TABLE BOOKINGS (
    BookingID INT PRIMARY KEY,
    ScreenID INT,
    UserID INT,
    FOREIGN KEY (ScreenID) REFERENCES SHOWS(ScreenID),
    FOREIGN KEY (UserID) REFERENCES CUSTOMER(UserID)
);

CREATE TABLE BOOKED_SEATS (
```

```
    SeatID INT PRIMARY KEY,
    SeatNumber VARCHAR(10),
    BookingID INT,
    ScreenID INT,
    FOREIGN KEY (BookingID) REFERENCES BOOKINGS(BookingID),
    FOREIGN KEY (ScreenID) REFERENCES SHOWS(ScreenID)
);

CREATE TABLE VENUE (
    VenueID INT PRIMARY KEY,
    VenueName VARCHAR(100),
    City VARCHAR(50)
);

CREATE TABLE EVENTS (
    EventID INT PRIMARY KEY,
    EventName VARCHAR(100),
    Category VARCHAR(50),
    VenueID INT,
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID)
);
```

```
CREATE TABLE EVENT_BOOKING (
    EventBookingID INT PRIMARY KEY,
    EventID INT,
    UserID INT,
    FOREIGN KEY (EventID) REFERENCES EVENTS(EventID),
    FOREIGN KEY (UserID) REFERENCES CUSTOMER(UserID)
);

CREATE TABLE EVENT_TICKETS (
    TicketID INT PRIMARY KEY,
    EventBookingID INT,
    FOREIGN KEY (EventBookingID) REFERENCES
EVENT_BOOKING(EventBookingID)
);


CREATE TABLE PAYMENTS (
    PaymentID INT PRIMARY KEY,
    Amount FLOAT,
    Status VARCHAR(20),
    TransactionID VARCHAR(100),
    BookingID INT,
    EventBookingID INT,
    FOREIGN KEY (BookingID) REFERENCES BOOKINGS(BookingID),
```

```
    FOREIGN KEY (EventBookingID) REFERENCES
EVENT_BOOKING(EventBookingID)
);

CREATE TABLE ADMIN (
    AdminID INT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Password VARCHAR(100),
    Role VARCHAR(50)
);

CREATE TABLE OTP_VERIFICATION (
    OtpID INT PRIMARY KEY,
    UserID INT,
    CreatedAt DATETIME,
    ExpiresAt DATETIME,
    FOREIGN KEY (UserID) REFERENCES CUSTOMER(UserID)
);
```

3.3 identify the primary keys and foreign keys used in schema

Primary Keys

- TheatreID
- MovieID
- AdminID
- UserID (Customer)
- ShowID
- BookingID
- SeatID
- PaymentID
- OTPID
- EventBookingID
- EventID

Foreign Keys

- Shows.MovieID → Movie(MovieID)
- Shows.TheatreID → Theatre(TheatreID)
- Bookings.ScreenID → Shows(ScreenID)
- Bookings.UserID → Customer(UserID)
- Booked_Seats.BookingID → Bookings(BookingID)
- Booked_Seats.ScreenID → Shows(ScreenID)
- Payments.BookingID → Bookings(BookingID)
- Payments.EventBookingID → Event-booking(EventBookingID)
- OTP_Verification.UserID → Customer(UserID)
- Event-booking.EventID → Events(EventID)
- Event-booking.UserID → Customer(UserID)